UNIVERSITY OF PASSAU

Faculty of Computer Science and Mathematics

Reverse- and Reengineering



Prof. Dr.-Ing. Christian Hammer

WS 2026/27

- **Lecture**: 2h/w
  - Wed 12:15–13:45
- **Tutorial**: 2h/w (choose one)
  - Tue 14–16, 16–18
- **Total: 6 ECTS**

- Reverse engineering third-party programs in processor-specific binary formats as well as intermediate codes. Effects of obfuscation techniques on this process. Transformation into a representation suitable for further processing, reconstruction of high-level data and control structures. Program analysis to improve program understanding. Automatic refactoring to improve program structure.
- *Recommended Skills*: Compiler Construction or Software Analysis (Lexing, Parsing, Intermediate Representation)

- The students are familiar with the techniques of analysis and refactoring of unspecified or outdated software systems, both with and without the availability of source code, in particular with the help of (semi-)automatic analysis.
- The students are able to independently create prototypical systems for the reconstruction of the program semantics and transformation into a software-technically improved system.
- The students are able to support software development work with the creation of smaller, domain-specific systems to reconstruct the program semantics or to improve the program structure of larger software systems.

- 2 dates, sign up via Campus Portal
- **Termin 1** (timeframe: 08.02. – 05.03.2027)
  - Sign up period: 16.11. – 15.12.2026, withdrawal until 11.01.2027
- **Termin 2** (timeframe 22.03. – 09.04.2025)
  - Sign up period: 01.03. – 18.03.2027, withdrawal until 18.03.2027
- Total 90 points, 90 min

NO guarantee, official: Prüfungssekretariat Uni Passau

- **Homework** not corrected
- But **required** to pass exam
- **Thinking things through on your own** makes you smarter;
  using LLM statistically leads to lower grades!

- Questions: Stud.IP (Tutorial)
- Larger discussions: Appointment with tutor

- Homework questions developed on blackboard together
  Bring your own ideas or there will not be a sample solution in the end

- Understand the concepts of Software Aging, Reengineering, Refactoring, Code Smells
- Explain the challenges of legacy systems and how they can be renewed
- Be able to understand and create simple intra- and interprocedural control flow graphs as well as program dependence graphs
- Understand and apply the concepts of program slicing
- Explain the difference between Maintenance and Reengineering

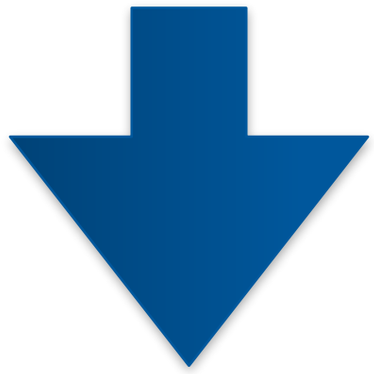
```
000024
000025  PROCEDURE DIVISION.
000026  0001-MAIN.
000027      INSPECT FUNCTION REVERSE(STR-1)
000028              TALLYING WS-LEN1 FOR LEADING SPACES.
000029      COMPUTE  WS-LEN = LENGTH OF STR-1 - WS-LEN1.
000030      DISPLAY WS-LEN.
000031      MOVE 1 TO I.
000032      MOVE WS-LEN TO J.
000033      PERFORM REV-PARA WS-LEN TIMES.
000034      DISPLAY STR-1.
000035      DISPLAY STR-2.
000036      GOBACK.
000037    REV-PARA.
000038      MOVE STR-1(J:1) TO STR-2(I:1).
000039      SUBTRACT 1 FROM J.
000040      ADD 1 TO I.
000041      EXIT.
****** ********************************** Bottom of Data ********************************
```

source: https://medium.com/@yvanscher/7-cobol-examples-with-explanations-ae1784b4d576

“Reengineering, also known as both renovation and reclamation, is the examination and alteration of a subject system to reconstitute it in a new form and the subsequent implementation of the new form.”

Chikofsky, Cross (1990)

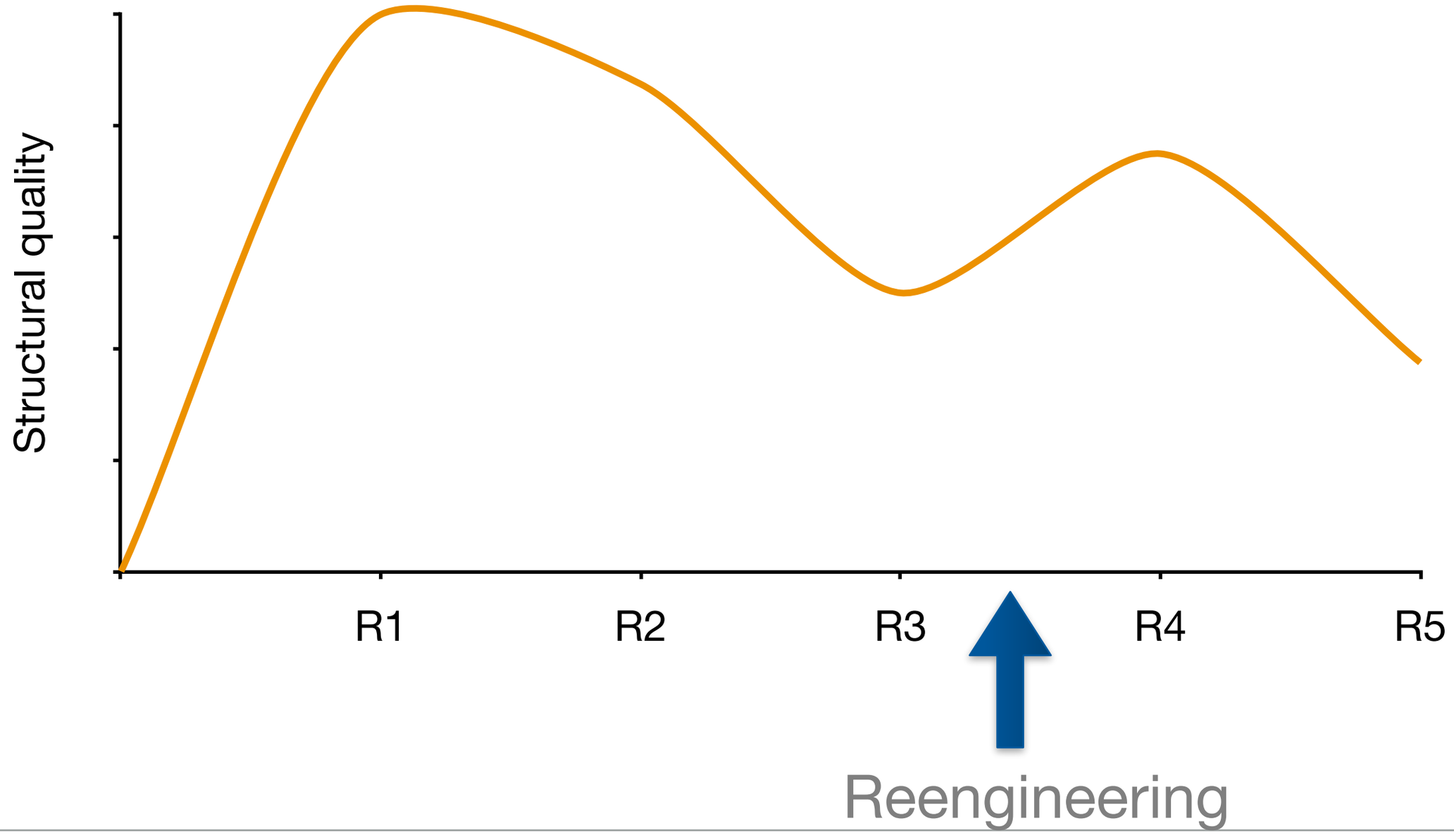
- Software requirements change over time
- Software is adapted to new requirements after release
- Adding functionality that is inappropriate to the architecture of the software

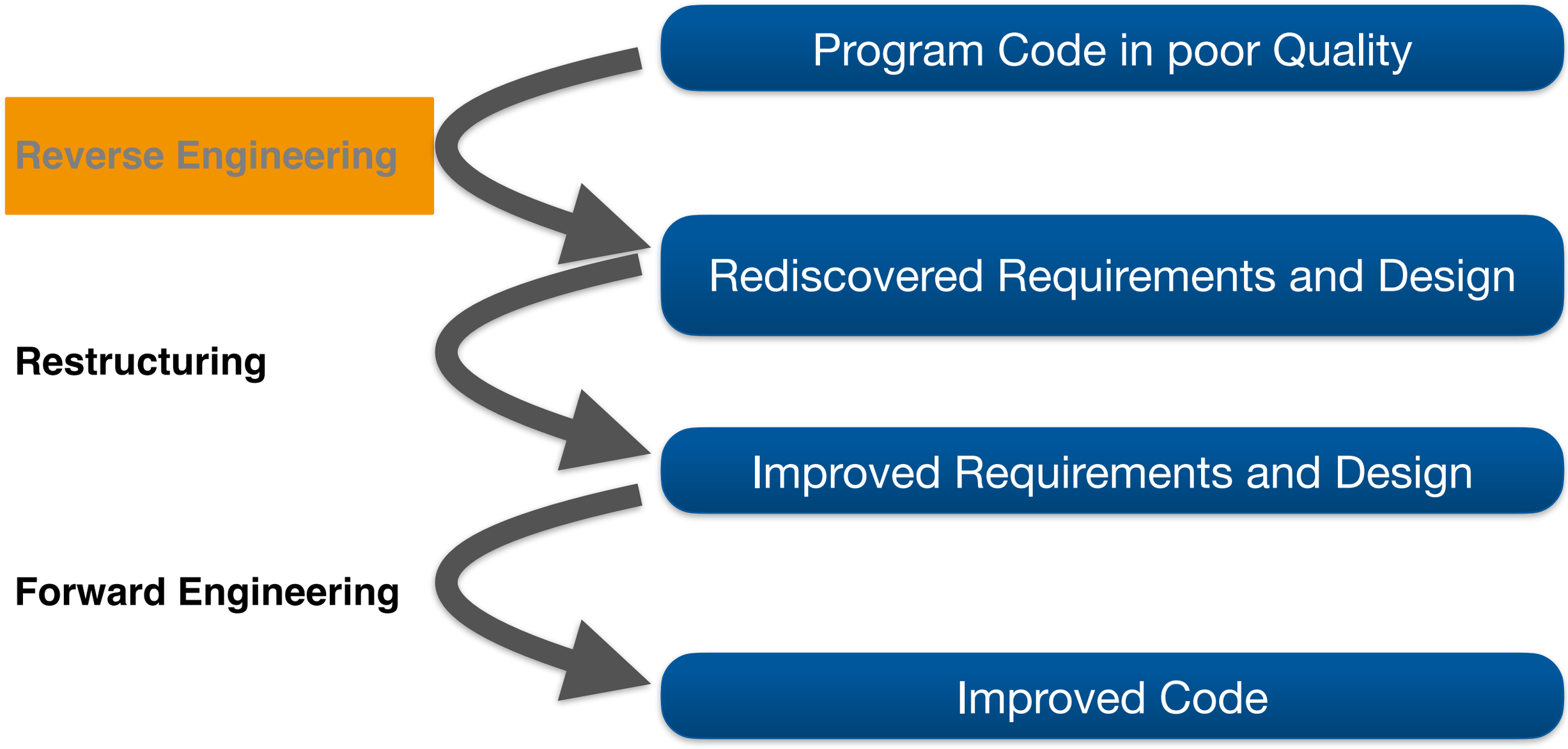


Software Quality Degradation

- The quality of software decreases with time (software aging):
  - Software becomes more error-prone
  - Loss of performance
  - Deterioration of maintainability
  - General deterioration of the architecture

- Aging process graph Fig 23-2 [Ludewig 587]

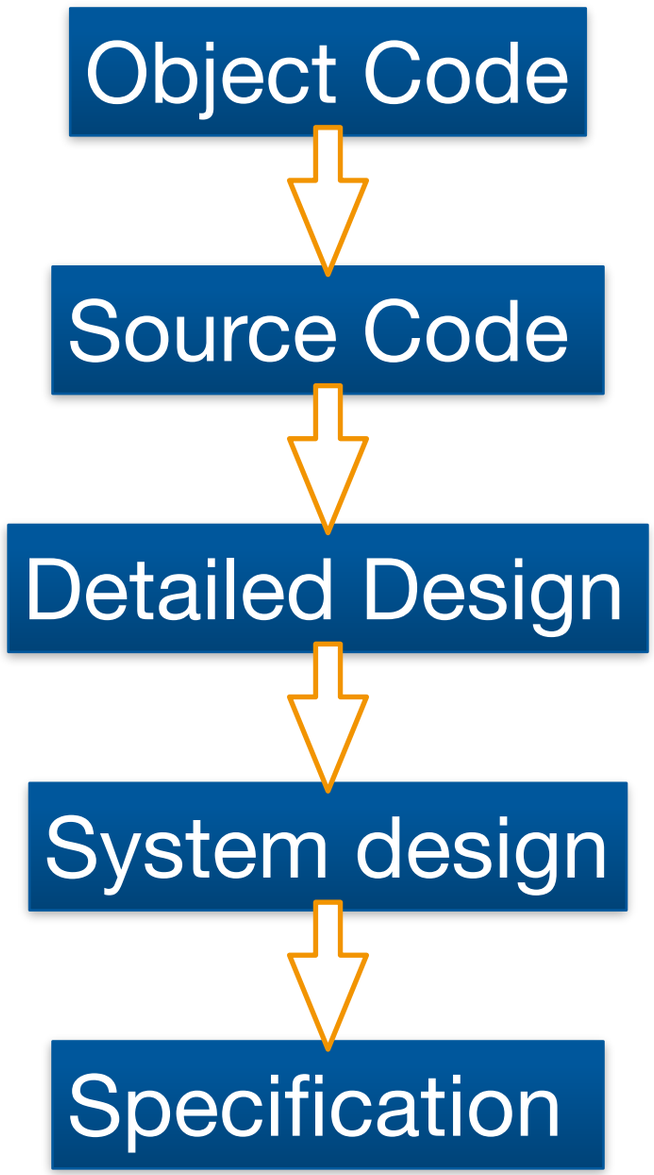



Program Code in poor Quality
Reverse Engineering
Rediscovered Requirements and Design
Restructuring
Improved Requirements and Design
Forward Engineering
Improved Code

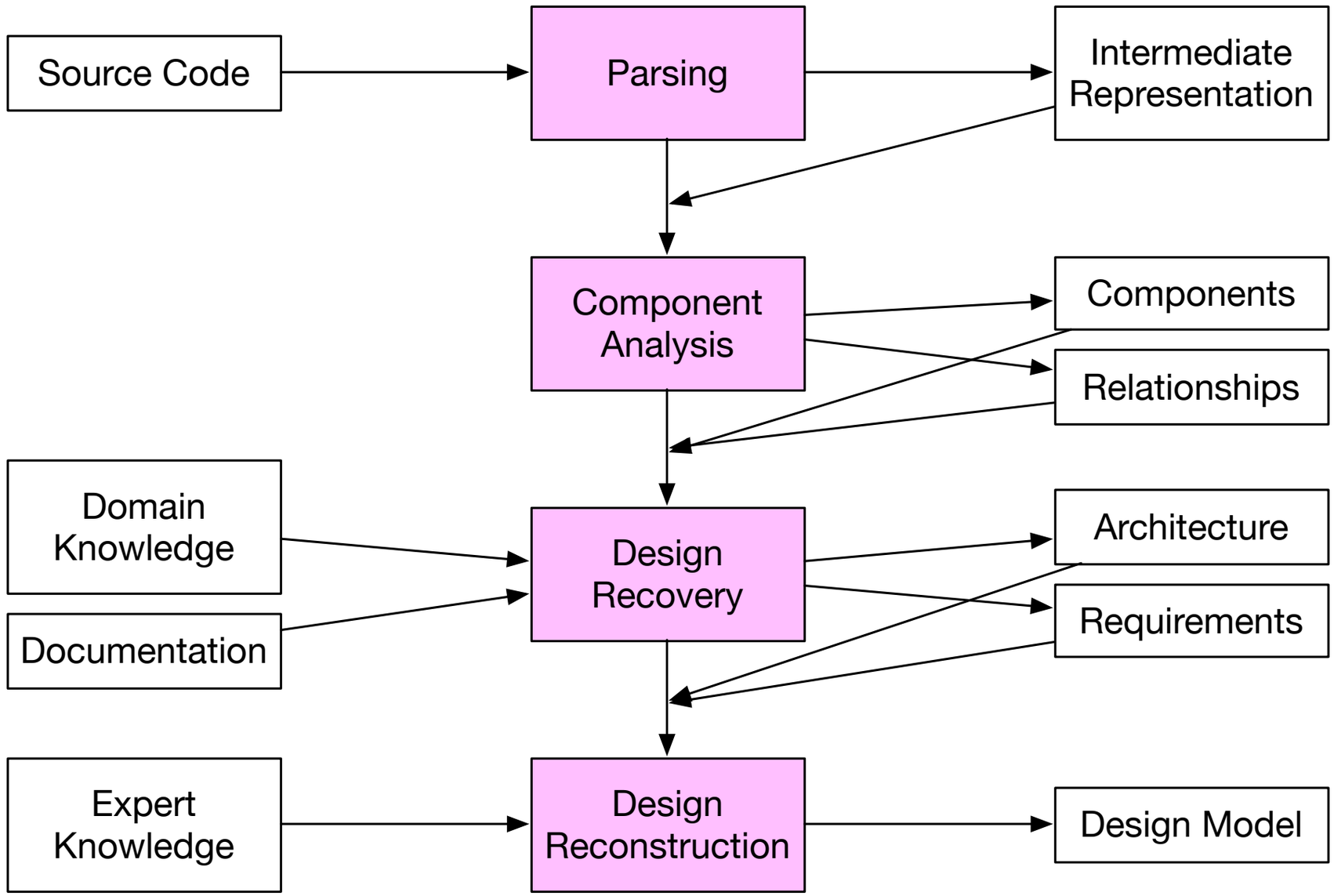
- Reverse Engineering:
  - Indicates a way to analyze a system
  - Identification of system components and their relationships
  - Presentation of the system in a more understandable form

- Goals of Reverse Engineering:
  - Facilitate understanding of the system
  - Create the basis for future software maintenance

- Reverse Engineering: Program Code —> Specification (Ludewig 591)



The Reverse Engineering Process



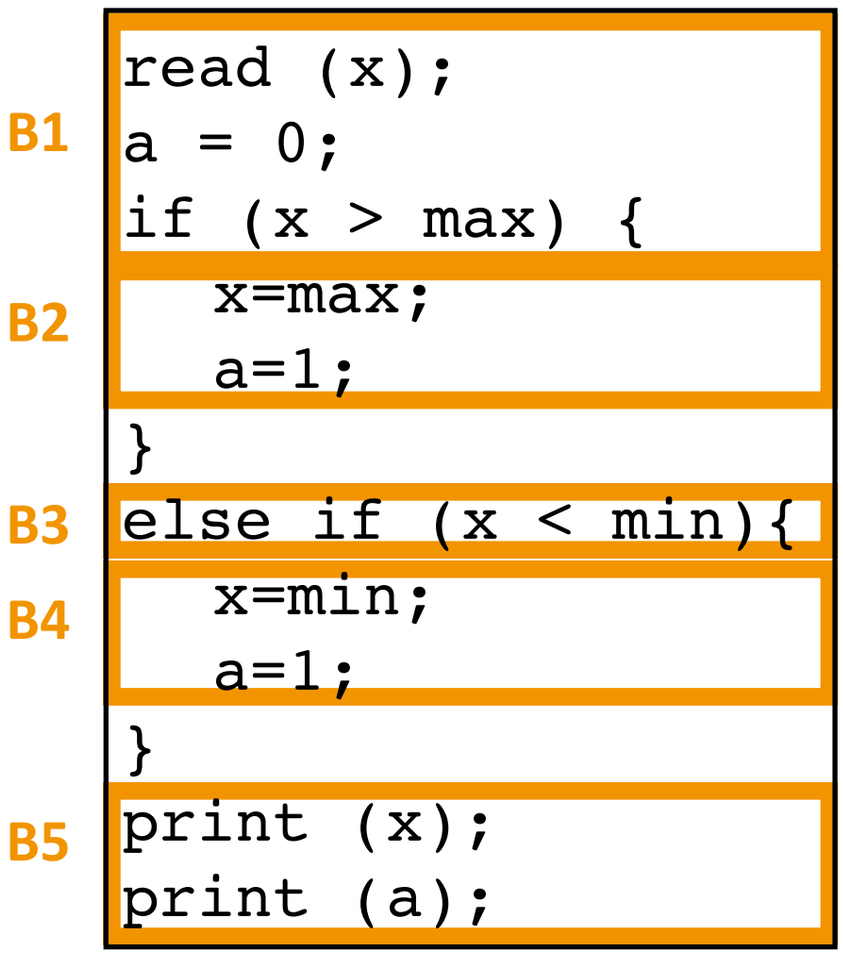
- Reverse Engineering Process (PsA 3.1.3)
- The control flow analysis makes statements about the order in which statements are executed in programs

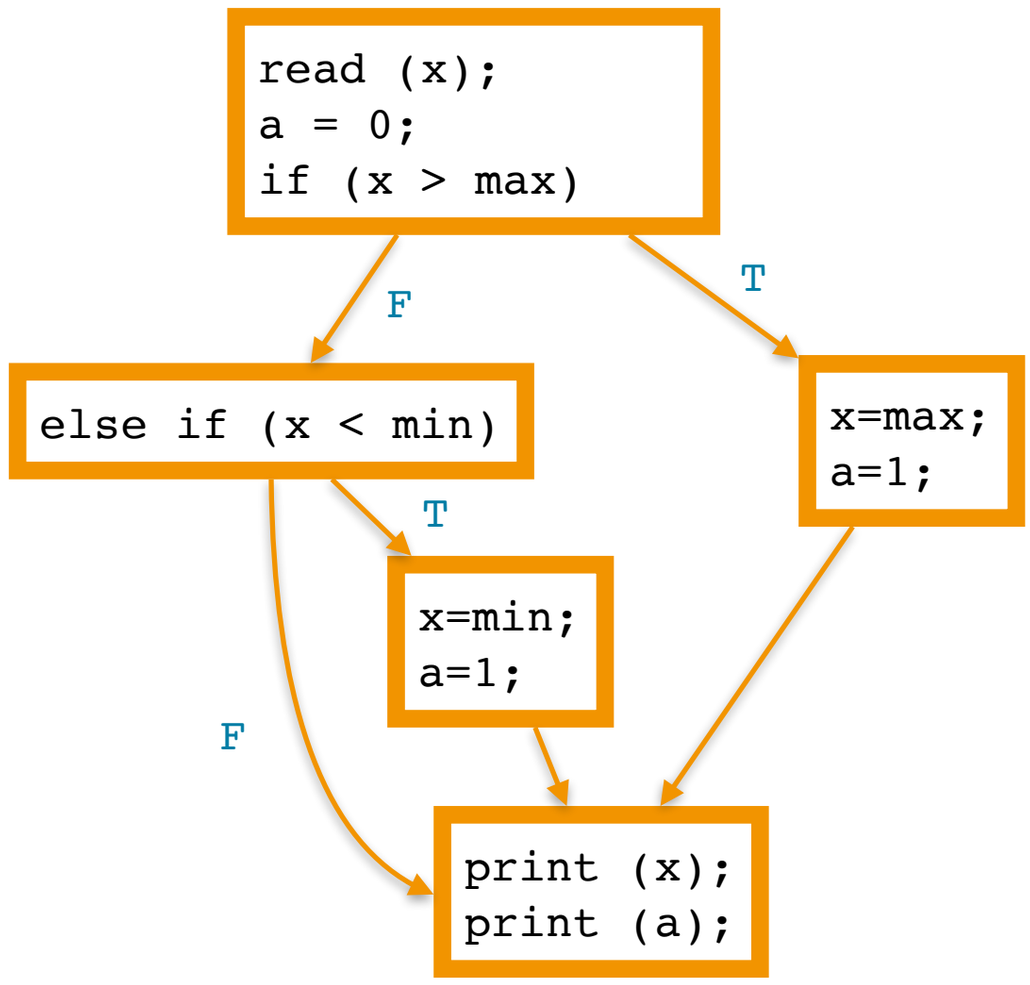
- A distinction is made between two types of control flow:
- **Intraprocedural** control flow:
  - Control flow analysis within a method
- **Interprocedural** control flow:
  - Control flow analysis between methods

A basic block is a chain of statements that is guaranteed to be executed one after the other for as long as possible

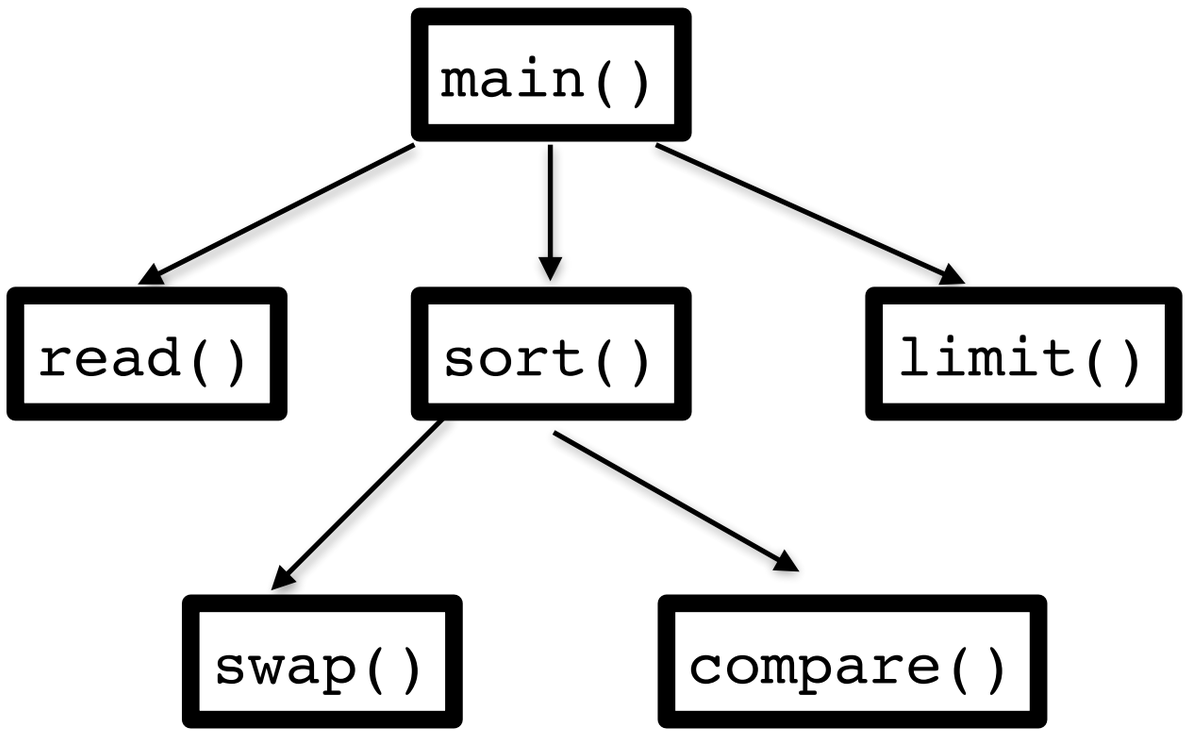
B1
```
read (x);
a = 0;
if (x > max) {
```
B2
```
   x=max;
   a=1;
}
```
B3
```
else if (x < min){
```
B4
```
   x=min;
   a=1;
}
```
B5
```
print (x);
print (a);
```

A basic block is a chain of statements that is guaranteed to be executed one after the other for as long as possible

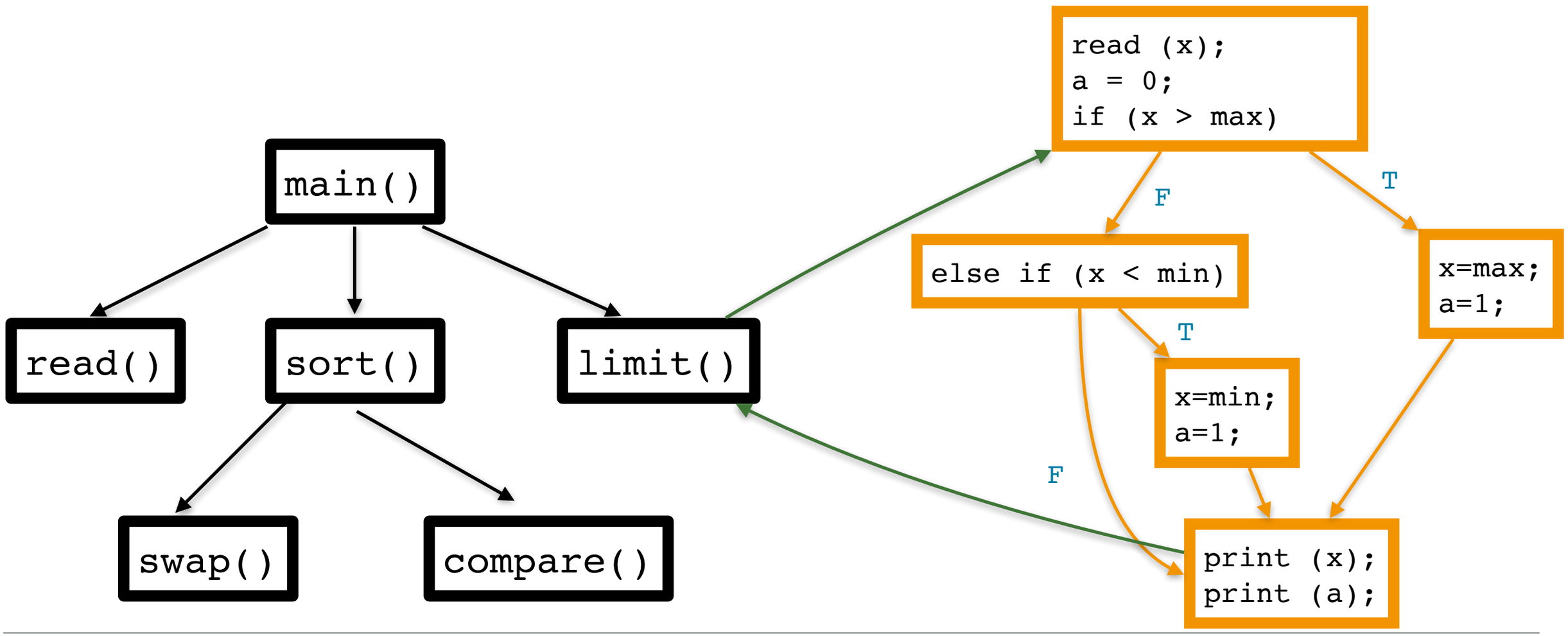




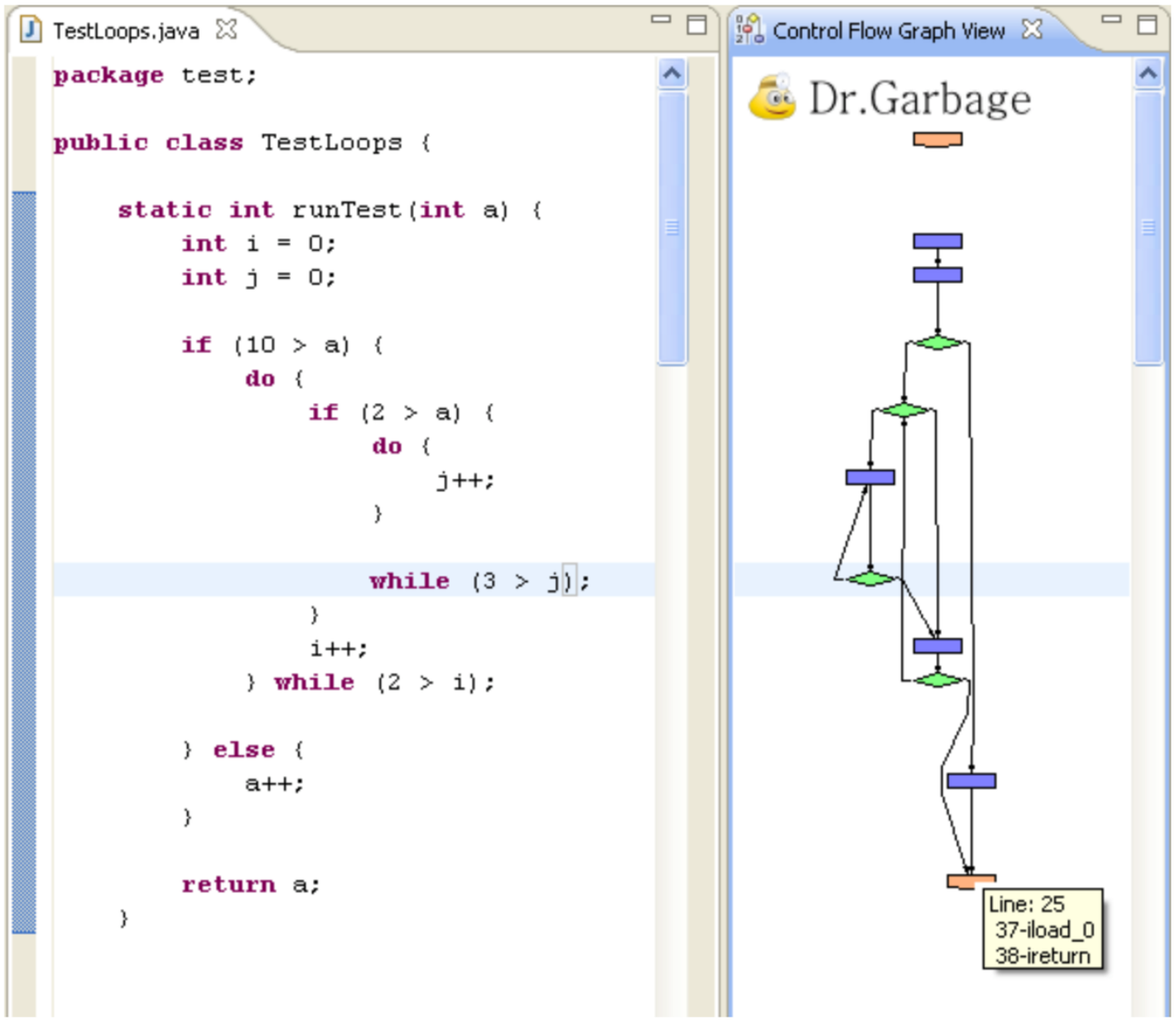
- Nodes in the graph are method names
- If a function ***a*** possibly calls a function ***b***, the graph will contain an edge from ***a*** to ***b***
- After a function is finished, it returns to the calling function



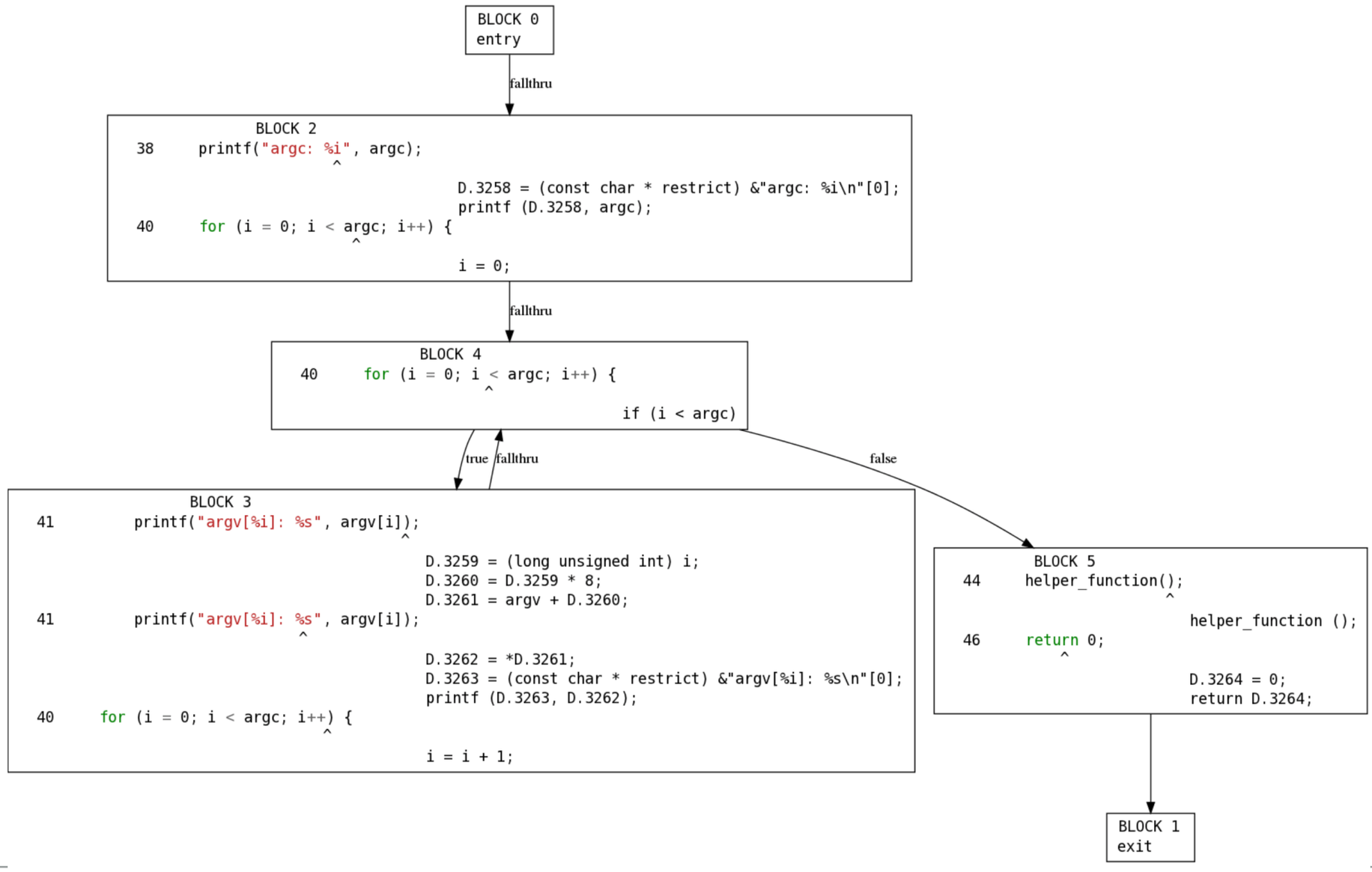
- An interprocedural control flow graph can also be combined with an intraprocedural control flow graph



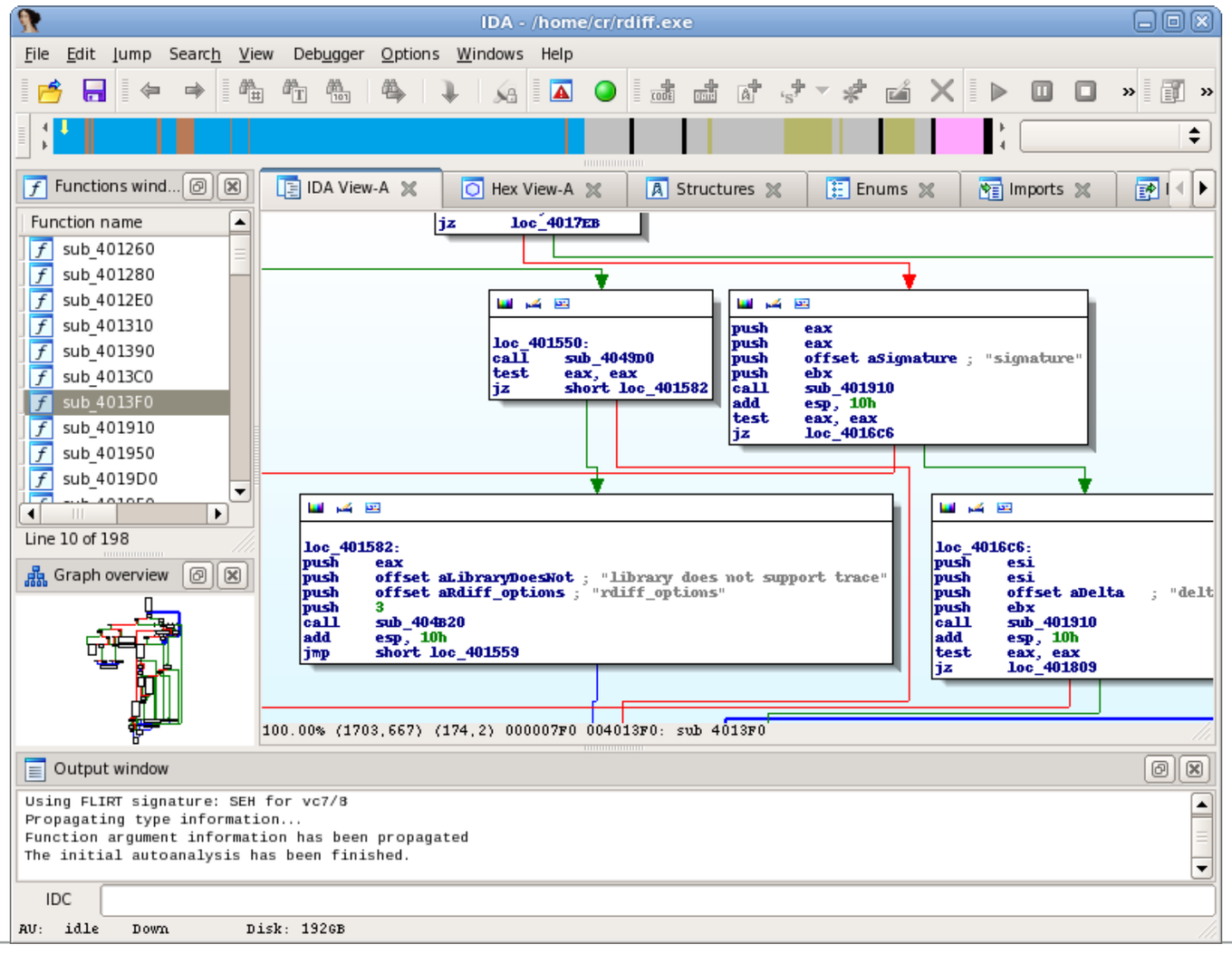
Sourcecode Visualizer is an Eclipse plugin that can render control flow graphs of Java code in real time



The GNU Compiler Collection offers the possibility to visualize the control flow of C programs



With IDA (Interactive Disassembler), compiled assembler code can be displayed as a control flow graph, among other things



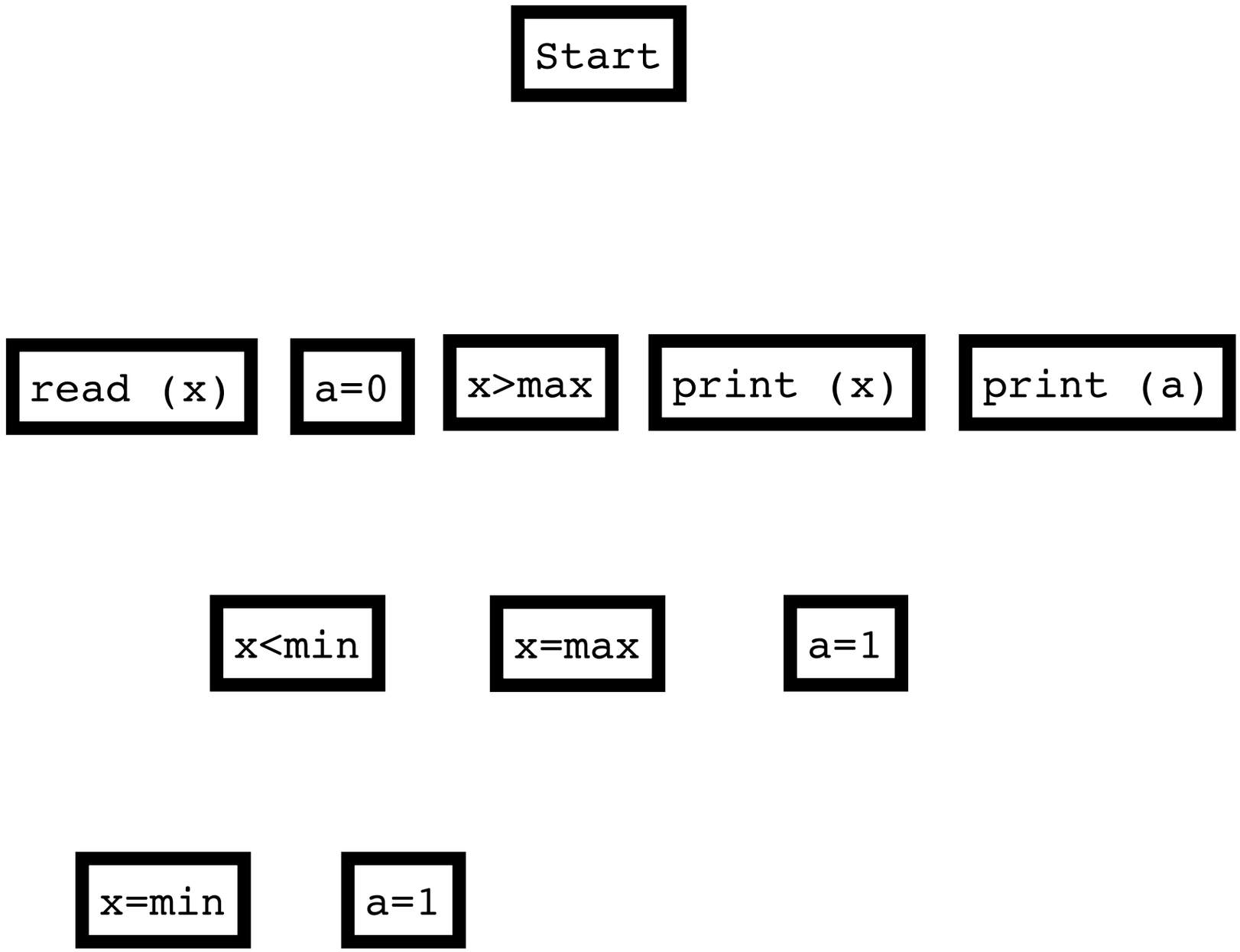
- A Program Dependence Graph (PDG) is intended to illustrate the dependencies between different statements within a method
- The order of statements is no longer visible in PDG
- A distinction is made between **control dependencies** und **data dependencies**

- **Control dependence:** a control dependency exists between two nodes `A` and `B` if node `A` determines the execution (if or how many times) of node `B`

- **Data dependence:** There is a data dependence between two nodes `A` and `B` if:
  - `B` uses a variable that was assigned a value at `A`, and
  - there is a path from `A` to `B` in the control flow graph without a value being assigned to this variable again

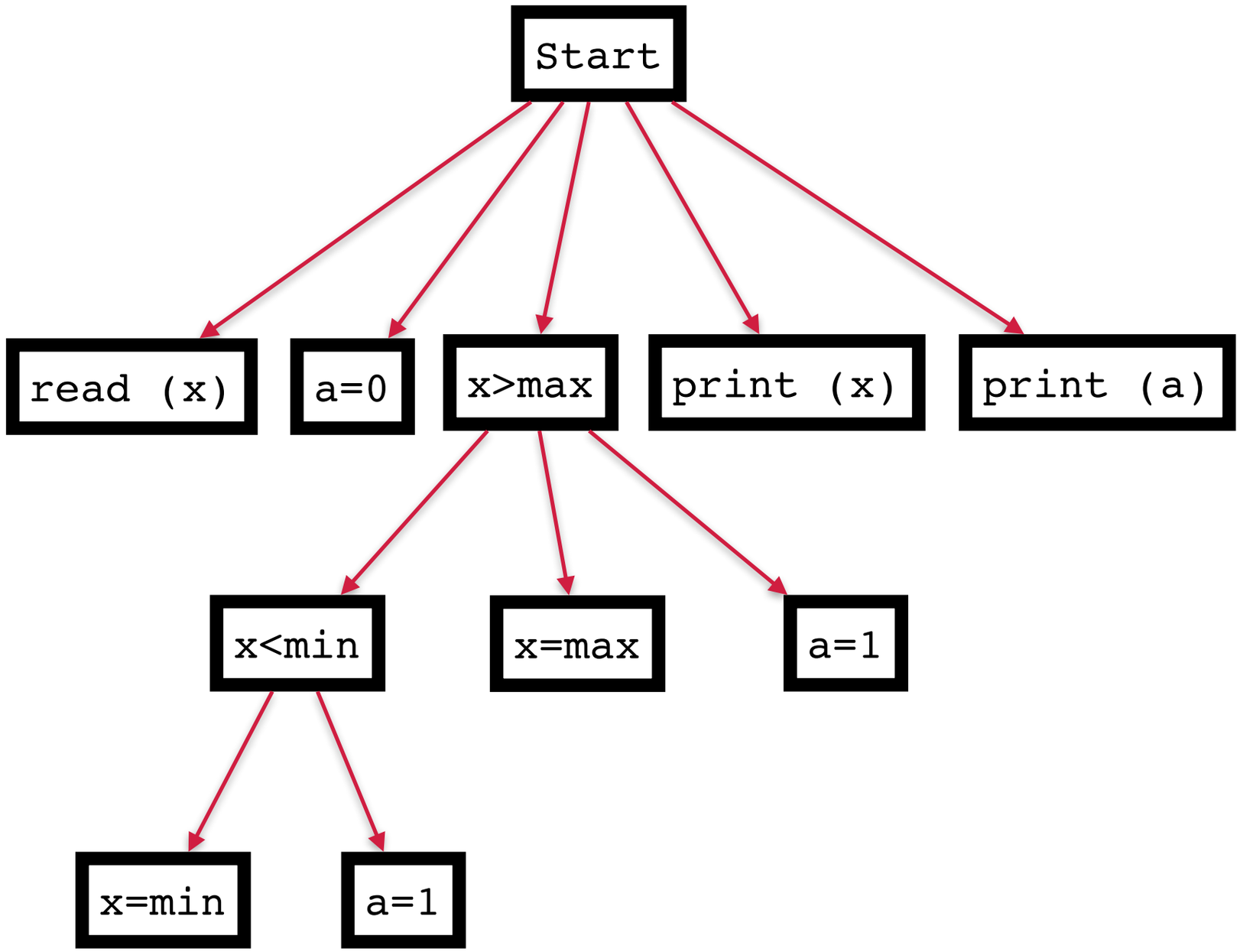
PDG Example - Nodes

```
read (x);
a = 0;
if (x > max) {
   x=max;
   a=1;
}
else if (x < min){
   x=min;
   a=1;
}
print (x);
print (a);
```



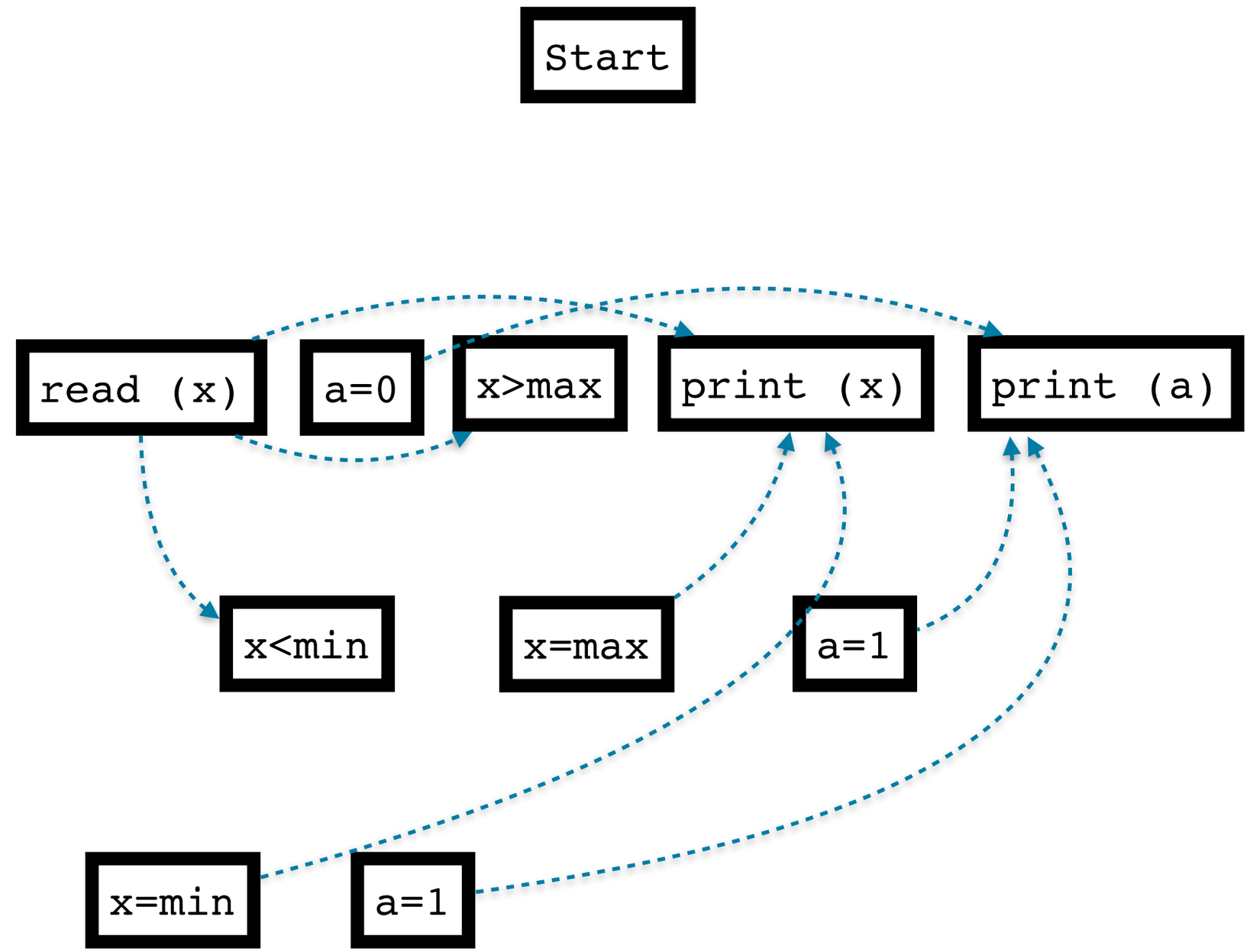
PDG Example - Control Dependencies

```
read (x);
a = 0;
if (x > max) {
   x=max;
   a=1;
}
else if (x < min){
   x=min;
   a=1;
}
print (x);
print (a);
```



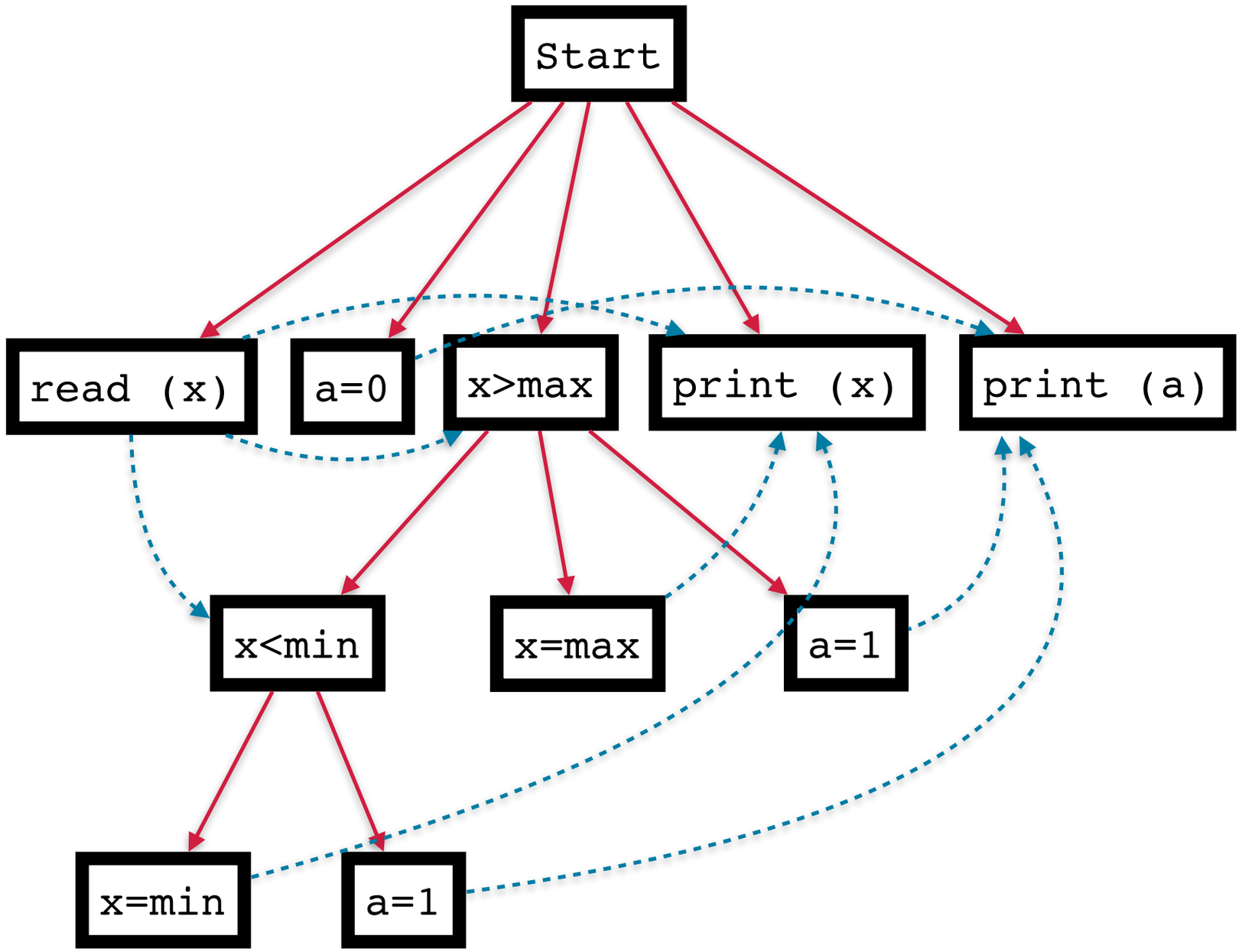
PDG Example - Data Dependencies

```
read (x);
a = 0;
if (x > max) {
   x=max;
   a=1;
}
else if (x < min){
   x=min;
   a=1;
}
print (x);
print (a);
```



PDG Example

```
read (x);
a = 0;
if (x > max) {
   x=max;
   a=1;
}
else if (x < min){
   x=min;
   a=1;
}
print (x);
print (a);
```



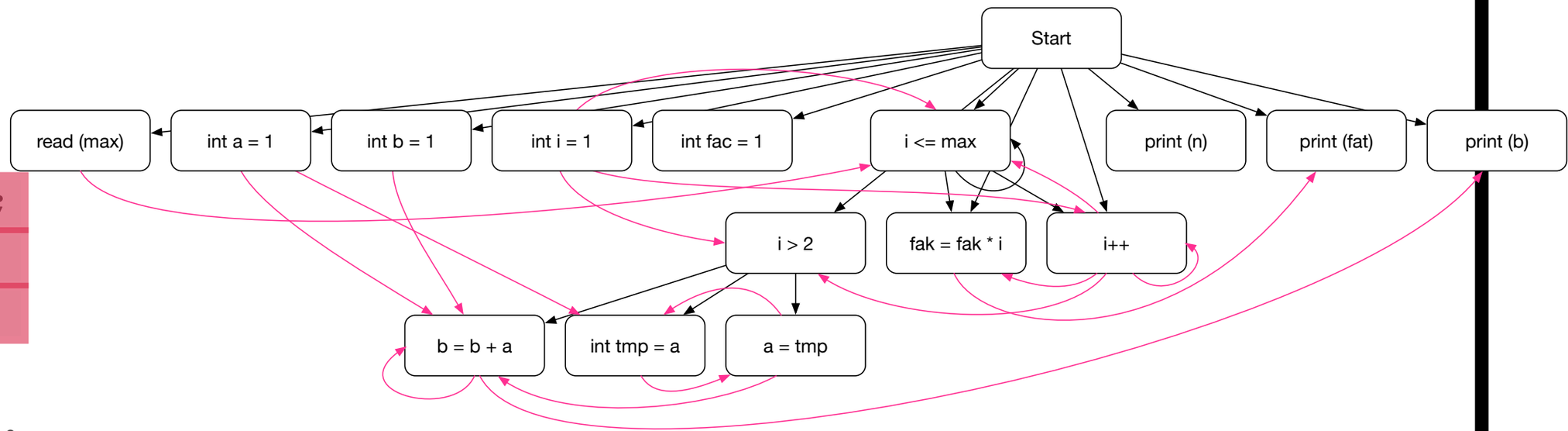
- Program Slicing is a type of static analysis used to illustrate the flow of data in programs
- A basic distinction is made between **Forward Slicing** and **Backward Slicing**
  - **Forward Slicing** describes which statements are influenced by a statement `s`
  - **Backward Slicing** describes which statements influence a statement `s`
- `s` is called the slicing criterion

- Slicing can be used during the reverse engineering process to better understand the program code
- Slicing answers the questions:
  - Which variables are affected by a particular variable
  - Which variables affect a particular variable

Forward Slicing

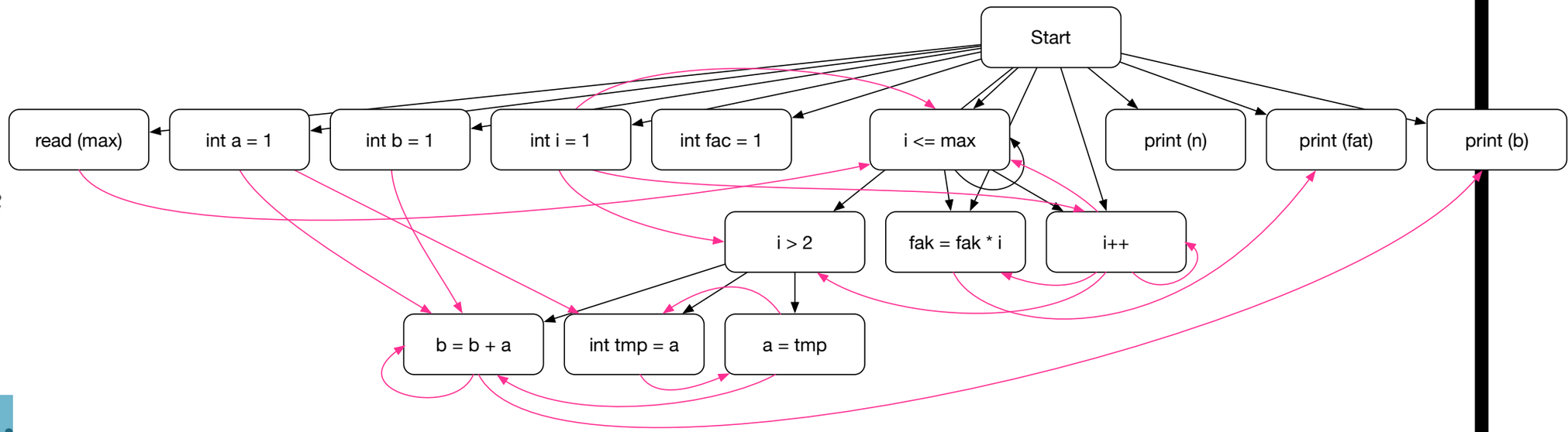
S

```
read (max);
int a = 1;
int b = 1;
int i = 1;
int fak = 1;
do{
  if (i > 2){
    int tmp = a;
    b= b + a;
    a = tmp;
  }
  fak = fak * i;
  i++;
}while (i <= max);
print (n);
print (b);
print (fak);
```



Backward Slicing

```
read (max);
int a = 1;
int b = 1;
int i = 1;
int fak = 1;
do{
  if (i > 2){
    int tmp = a;
    b= b + a;
    a = tmp;
  }
  fak = fak * i;
  I++;
}while (i <= max);
print (n);
print (b);
print (fak);
```



S

Further criteria in slicing algorithms:

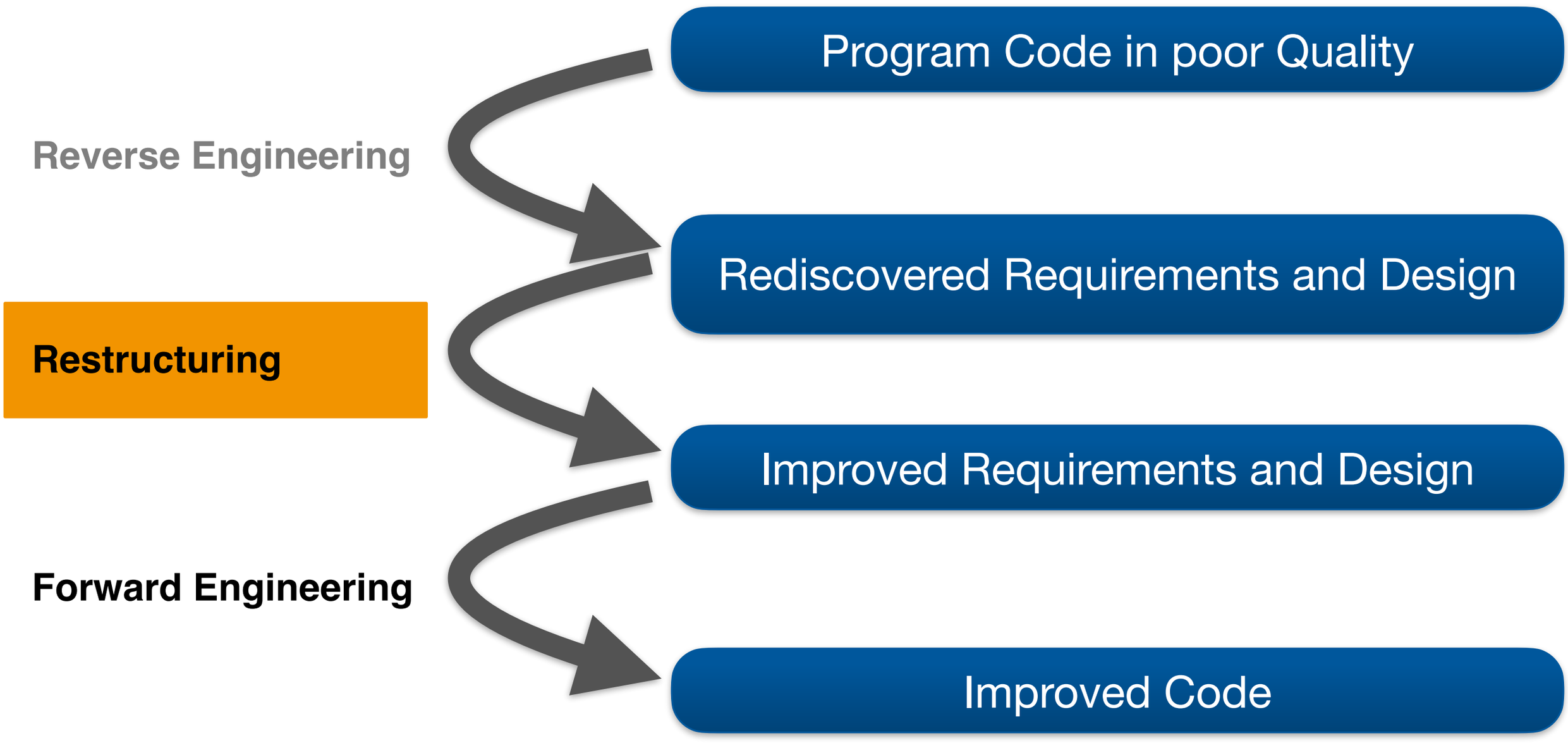
- **Static vs Dynamic:** static slicing considers only information available at compile time, while dynamic slicing also includes runtime information
- **Underlying data flow analysis:** slicing is based on an underlying analysis algorithm
  - Slices vary depending on the underlying algorithm
- **Considered data unit:** determines whether the slicing criterion refers to a single variable, a set of variables, or an entire row

In practice, a slicing algorithm is specified for a particular programming language. Among other things, the following must be considered here:

- Is interprocedural slicing required?
- Is it possible to jump to any position in the program code? (e.g. **GOTO** instruction)?
- Late binding of the programming language e.g. by polymorphism
- Concurrency

- Post-documentation:
  - Presentation of existing code in a clearer form
  - Attempt to extract higher language constructs from archaic code (e.g. loops, parameters, abstract data types)
- Architectural rediscovery:
  - Derivation of software architecture from existing program code
  - Requires knowledge of software architectures and programming language used as well as successful post-documentation

- Loss of information (e.g. methods and attribute names) during the compilation process => interpretation and expert knowledge of the code during the analysis becomes necessary
- Static analysis cannot always make statements about whether code is actually executed in practice
- The reverse engineering process can only be partially automated due to the analyst's own high level of effort


Program Code in poor Quality
Reverse Engineering
Rediscovered Requirements and Design
Restructuring
Improved Requirements and Design
Forward Engineering
Improved Code

Restructuring:

- Transformation of a system from one form into another
- The new form of the system has the same semantics and the same external behavior as the old form

- Refactoring is part of the restructuring process
- Improve the internal structure of code while accurately maintaining the external behavior of the program
- Intensive testing of the refactorized program to avoid errors due to Refactoring (→ regression testing)
- Refactoring is divided into two parts:
  - code refactoring
  - architecture refactoring

- “Code smell” describes poor quality or poorly structured program code
- Typical examples for “Code smell”:
  - Unused functionality
  - Too much or too little functionality in a method or class
  - Code duplicates
  - Parameter list too long
  - Message chains too long
  - Too many levels of nesting (e.g. nested if-statements)

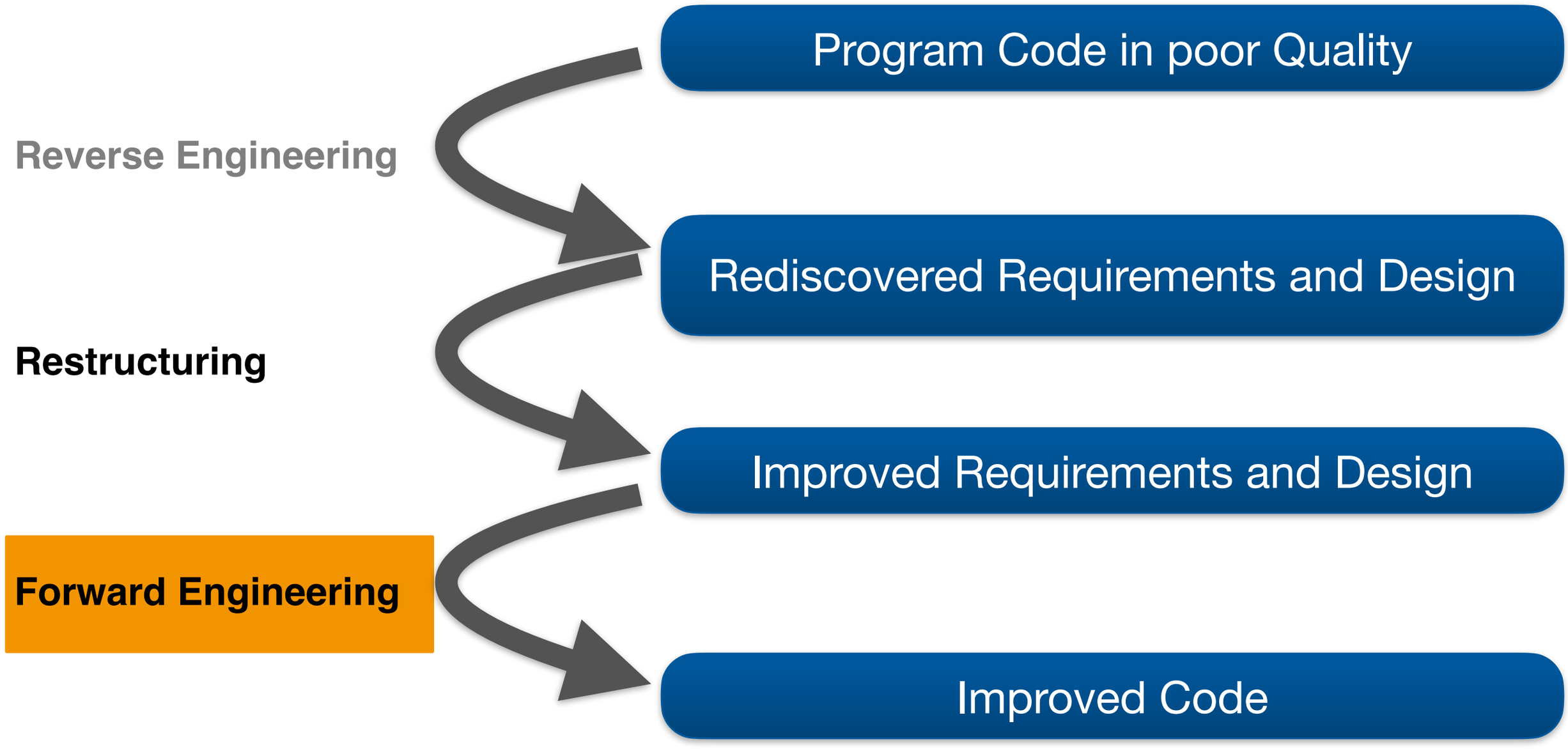
Code smell requires code refactoring

- Methods for code refactoring are for example:
  - Internal improvement of methods
  - Restructuring of the data organization (e.g. combining several primitive variables into a common object)
  - Splitting of complex conditional expressions

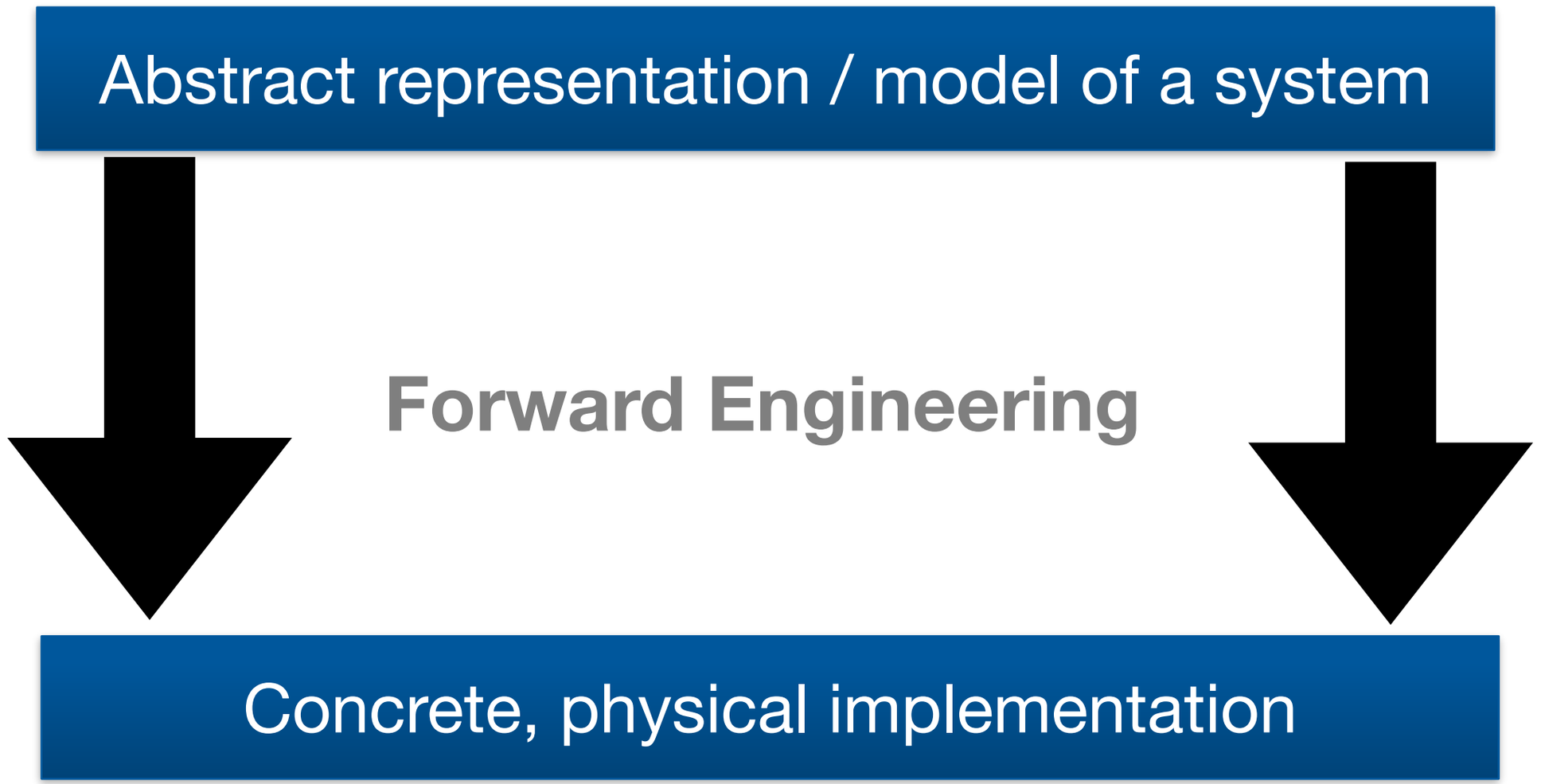
- “Architecture smell” describes vulnerabilities at the software architecture level
- Typical examples for “Architecture smell”:
  - Cyclic use of classes or unused classes
  - Parallel or too deep inheritance hierarchies
  - Packages or components too large or too small
  - break through/violation of stratification

Architecture smell requires architecture refactoring

- Denotes improvement of structure of software architecture
- Methods for code refactoring are e.g.:
  - Change of inheritance hierarchy
  - Split or merge of packages
  - extreme case: change of software architecture
- Architecture refactoring cannot be automated and requires human expert knowledge


Program Code in poor Quality
Reverse Engineering
Rediscovered Requirements and Design
Restructuring
Improved Requirements and Design
Forward Engineering
Improved Code

- Explanation of term “Forward Engineering” [Ludewig 589]



- Definition of Reengineering vs. Maintenance [Ludewig 585 Introduction]
- Maintenance: Adaptation of the software according to the customer's wishes
  - Enhancement of the software
  - Software correction
  - Change of specifications

Reengineering:
Serves the developer

VS

Maintenance:
Serves the customer

- Properties of legacy systems:
  - Legacy systems leverage obsolete programming languages, platforms and operating systems
  - Often poorly documented
  - Often very large code base
  - Lack of knowledge of the employees in the company about handling and architecture of legacy system

- Nevertheless, legacy systems oftentimes perform critical tasks in a company

- There are three basic procedures for renewing legacy systems:
  - **Packing:** a new software is designed as a shell that provides a modern interface to the outside world, but communicates internally with the legacy software
  - **Migration:** it is possible to renew software step by step
    - To do this, you divide them into components and renew one component after the other
  - **Replacement in one fell swoop:** You develop the software completely from scratch while still using the old software during the development phase

- “Software Engineering” - Jochen Ludewig & Horst Lichter
- “Software Aging” - David Large Parnas
- “Programmverstehen und statische Analysetechniken im Kontext des Reverse Engineering und der Qualitätssicherung” - Marcel Bennicke & Heinrich Rust
- “Refactoring: Improving the Design of Existing Code” - Martin Fowler